
Kubernetes Static Pods

📦 What Are Static Pods?

Static Pods are **special pods managed directly by the kubelet** rather than the Kubernetes API server.

Unlike regular pods:

- They are **node-specific**
- Not managed by **Deployments, ReplicaSets, or controllers**
- Often used for **critical system components**, like control plane services, or for **manual debugging/bootstrapping**

🧠 Key Concepts

1️⃣ Created by the Kubelet

- The **kubelet** is the agent running on each Kubernetes node.
- **Static Pods are defined locally** on a node in YAML files.
- The kubelet continuously watches the static pod directory and **creates/deletes pods automatically** based on file changes.
- They **cannot be created using kubectl apply**.
- **kubectl only observes them**, because the kubelet **mirrors** them to the API server.

2️⃣ Where Do You Define Static Pods?

- Static Pod YAML files are usually placed in:

/etc/kubernetes/manifests/

- The kubelet reads any YAML in this directory.
- Example:

/etc/kubernetes/manifests/nginx.yaml

> **Tip:** You can configure the kubelet to use a custom static pod directory via the --pod-manifest-path flag.

3️⃣ Kubelet Watches the Directory

- **Automatic creation**: Any YAML file added to the directory → kubelet runs the pod.
- **Automatic deletion**: Deleting the YAML → kubelet deletes the pod.
- **Automatic updates**: Modifying the YAML → kubelet restarts the pod with updated spec.

> ✅ This is a **node-level mechanism**; no scheduler or controller is involved.

4️⃣ No Controllers Involved

Static Pods **do not use**:

- Deployments
- ReplicaSets
- StatefulSets
- Horizontal Pod Autoscalers

> They **run exactly as specified**, on the node where the YAML file exists.

🔧 Example: Static Pod YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: static-nginx
  labels:
    role: static
spec:
  containers:
  - name: nginx
    image: nginx:1.25
    ports:
    - containerPort: 80
```

Steps to deploy:

1. Save the file as:

/etc/kubernetes/manifests/static-nginx.yaml

2. Kubelet automatically creates the pod on that node.

🔍 How to Identify Static Pods

- Use kubectl get pods:

```
kubectl get pods -A -o wide
```

- **Static Pods characteristics**:

Feature	Observation

Namespace	Usually kube-system
Pod name	<pod-name>-<node-name>
Controller	None (OwnerReferences point to kubelet)

- Example:

```
static-nginx-node1   Running   1/1   node1
```

> **Tip:** Static Pods are visible in Kubernetes API as **"mirror pods"**, but they are actually managed by the kubelet.

🧩 Use Cases

1. **Control plane components**
   - kube-apiserver, kube-scheduler, kube-controller-manager
2. **Critical node-level services**
   - Logging, monitoring agents, custom node services
3. **Cluster bootstrapping or advanced debugging**
   - Early cluster setup before API server is fully functional

---

🛑 Limitations

Limitation	Explanation
No replication	Each static pod runs **only on the node where YAML exists**

No self-healing	Pods are **not managed by controllers**; if they fail, the kubelet only restarts them
Manual per-node configuration	Must be **added to each node** separately; no cluster-wide deployment

---

🔚 Summary Table

Feature	Static Pods
Managed by	Kubelet only
Created via	Local YAML file (/etc/kubernetes/manifests)
Scheduling	Fixed to one node
Use Cases	System components, node-level services
Replication	❌ No
Self-healing	❌ No

🧪 Lab Exercise: Deploy a Static Pod

Step 1: Connect to a Node

ssh ubuntu@node1

Step 2: Create the Static Pod YAML

sudo vi /etc/kubernetes/manifests/static-nginx.yaml

Paste:

```
apiVersion: v1
kind: Pod
metadata:
  name: static-nginx
  labels:
    role: static
spec:
  containers:
  - name: nginx
    image: nginx:1.25
    ports:
    - containerPort: 80
```

Step 3: Wait for Kubelet to create the Pod

kubectl get pods -A -o wide

- You should see static-nginx-<node-name> running in kube-system.

Step 4: Test Pod Access

kubectl exec -it static-nginx-<node-name> -- /bin/sh

- Inspect container logs or run test commands.

Step 5: Delete the Pod

sudo rm /etc/kubernetes/manifests/static-nginx.yaml

- Kubelet will **automatically remove the pod** from the node and the API server.

⚡ Key Takeaways

- Static Pods are **kubelet-managed**, node-specific pods.
- Ideal for **critical system components** or **early cluster bootstrapping**.
- **Not suitable** for workloads requiring scaling or high availability.
- **Mirror Pods** in the API server allow visibility, but actual control is **local** to the node.

🧠 Bonus Notes

- Static Pods are **used internally by kOps, kubeadm**, and other cluster provisioning tools for control plane components.
- Can be combined with **taints/tolerations** for node-specific critical services.
- Useful for **debugging cluster startup issues** before the scheduler is fully functional.

Perfect question! Let's break it down **clearly and precisely**, because the distinction can be subtle:

1️⃣ Pods Managed by DaemonSets

DaemonSet: Ensures that **one copy of a pod runs on every node (or a subset of nodes)**.

- Managed by: **Kubernetes Controller Manager** via the **DaemonSet controller**
- How it works: You define a DaemonSet, and the **controller automatically creates pods on nodes** according to labels/taints/affinities.
- Pod creation: **Through the API server**, not directly by kubelet.
- Example:

kubectl get daemonsets -A

- Use case: Logging agents (Fluentd), monitoring agents (Prometheus node exporter), network plugins (Calico, Cilium)

✅ Key point: Pods created by DaemonSets are **controller-managed**, automatically reconciled if deleted.

2️⃣ Pods Managed Directly by Kubelet (Static Pods)

Static Pods: Special pods **defined locally on a node**, not through the API server.

- Managed by: **Kubelet on the node**
- How it works:
  - YAML files placed in /etc/kubernetes/manifests/ (or --pod-manifest-path directory)
  - Kubelet automatically creates the pod
  - A **mirror pod** is created in the API server so it's visible via kubectl, but **the API server does NOT manage it**
- Pod creation: **Node-local via kubelet**, **not through kubectl or scheduler**
- Example:

ls /etc/kubernetes/manifests/

kubectl get pods -A -o wide

- Use case: Control-plane components (kube-apiserver, kube-scheduler, etc.), critical node-level services

✅ Key point: Static pods are **node-specific**, **no replication**, **no scheduler involved**.

3️⃣ Pods Managed via Kubectl / Deployments / ReplicaSets

Normal pods that you create with kubectl are typically **managed by controllers**:

- **kubectl apply -f pod.yaml** creates a pod, but:
  - If it's a **raw Pod YAML**, **kubectl just tells the API server to create it**.
  - If the pod is **deleted manually**, it's gone (no controller to restore it).
- **Better practice:** Use **Deployments**, **StatefulSets**, or **ReplicaSets**, which automatically manage pods:
  - If pods crash, the controller **reschedules** them
  - Scaling is automatic
  - Scheduler determines **which node** the pod runs on

Example:

kubectl apply -f deployment.yaml

kubectl get pods -o wide

- Use case: Application workloads, scalable services

✅ Key point: These pods are **API server & controller managed**, flexible, and can move between nodes.

Quick Comparison Table

Pod Type	Managed By	Creation Method	Scheduler Used?	Node-specific?	Reschedules if Deleted?

Static Pod	Kubelet	YAML in /etc/kubernetes/manifests	❌ No	✅ Yes	Kubelet restarts only
DaemonSet Pod	DaemonSet Controller	API Server	✅ Yes	Depends on DaemonSet spec	✅ Yes, controller recreates
Normal Pod (kubectl)	API Server (with/without controller)	kubectl apply / Deployment	✅ Yes	❌ Flexible	Depends on controller (Deployment: yes, raw pod: no)

💡 **Teaching Note:**

- **Static pods** → kubelet-managed, node-specific, no scheduler
- **DaemonSet pods** → controller-managed, auto-replicated across nodes
- **kubectl/deployment pods** → API server-managed, scheduler assigns nodes, flexible and scalable